

```

1  r"""prelude-doctests.py -- every prelude name explained and demonstrated.
2
3  The one file in the course allowed an import, because the prelude IS its subject. One entry per name, in
4  prelude.py's own order. Each entry is a short explanation -- what it does, what the arguments mean, when
5  to reach for it, and the gotcha if there is one -- followed by several doctest examples, so the whole
6  reference is executable:
7
8      python3 prelude-doctests.py          # runs every example
9
10 Two conventions to load before reading. First, argument order is Haskell's: the FUNCTION comes first, the
11 data second -- map_(f, xs), sortOn(key, xs). Second, inside a fold the combiner's own arguments differ by
12 direction: foldl's f takes (accumulator, element); foldr's f takes (element, accumulator). When a fold
13 misbehaves, check that order first.
14
15 ===== 1. combinators =====
16
17 id -- the identity function: returns its argument unchanged. Sounds useless; is everywhere. It is the "do
18 nothing" you pass where a transformation is required -- a sort key meaning "the value itself", the seed
19 of compose.
20 >>> id(42)
21 42
22 >>> sortOn(id, [3, 1, 2])
23 [1, 2, 3]
24
25 const -- freezes a value into a function: const(x) returns a function that ignores its argument and
26 always answers x. Use it to fill a callback slot with a constant.
27 >>> five = const(5)
28 >>> five("anything"), five(99)
29 (5, 5)
30 >>> map_(const('*'), [1, 2, 3])
31 ['*', '*', '*']
32
33 flip -- swaps the two arguments of a function. An adapter: the function you have takes (a, b), the slot
34 you are filling supplies (b, a). This is how the prelude derives foldr from foldl.
35 >>> sub = lambda a, b: a - b
36 >>> sub(10, 1), flip(sub)(10, 1)
37 (9, -9)
38 >>> flip(pow)(3, 2)          # pow(2, 3)
39 8
40
41 How it works: flip(f) returns a new function that hands its two arguments to f in the opposite order.
42 Nothing is computed at flip time -- it is pure plumbing, applied when the data arrives in the "wrong"
43 order for the function you already have.
44 >>> pair = lambda a, b: (a, b)
45 >>> flip(pair)(1, 2)
46 (2, 1)
47
48 Why it earns its place: the prelude's own foldr is the showcase. foldl's combiner takes (accumulator,
49 element); a right fold's combiner takes (element, accumulator). Rather than build a second fold machine,
50 foldr = foldl of the FLIPPED combiner over the reversed list -- one adapter turned an existing engine
51 into its mirror image.
52 >>> f = lambda x, acc: '(' + x + acc + ')'
53 >>> foldr(f, "abc", "") == foldl(flip(f), list(reversed("abc")), "")
54 True
55
56 Remember it as: when argument order is the only thing wrong, adapt -- don't rewrite.
57
58 curry -- splits a two-argument function into two one-argument stages, so you can supply the first
59 argument now and the second later.
60 >>> add = lambda a, b: a + b
61 >>> curry(add)(1)(2)
62 3
63 >>> add_ten = curry(add)(10)
64 >>> add_ten(5)
65 15
66
67 How it works: read the three lambdas inside-out, then watch a call peel one layer at a time. Each call
68 binds ONE argument by closure -- after curry(add)(10), the 10 lives in the returned function's captured
69 environment, and that half-applied function is a value you can name, store, and pass.
70 >>> curry(add)(10)(5)
71 15
72 >>> add_ten = curry(add)(10)
73 >>> add_ten(5), add_ten(90)
74 (15, 100)
75
76 Why it earns its place: the prelude's pipelines -- map_, filter_, find, compose -- all want ONE-argument
77 functions in their function slot, while real logic usually takes two ("multiply by k", "is x greater than
78 n?"). Currying is the adapter: close over the configuration argument now, hand the pipeline the unary
79 stage it wants.
80 >>> gt = curry(lambda n, x: x > n)
81 >>> filter_(gt(10), [4, 11, 8, 40])
82 [11, 40]
83 >>> scale = curry(lambda k, x: k * x)
84 >>> map_(scale(3), [1, 2, 3])
85 [3, 6, 9]
86
87 The Haskell connection: in Haskell EVERY function is curried automatically -- add :: Int -> Int -> Int
88 really means Int -> (Int -> Int), so add 10 is ordinary code and map (add 10) xs falls out for free.

```

```

89 Every arrow in a Haskell type is another layer of this doll.
90
91 Versus partial: both bind early. partial binds any number of arguments, all at once, when you have them;
92 curry restructures the function so binding happens one call at a time, decided later by whoever holds it.
93 In Python, partial is the cheaper everyday tool; curry is the pipeline-shaper and the reading key for
94 Haskell signatures.
95
96 Remember it as: curry turns one two-argument function into two one-argument functions -- and one-argument
97 functions are the only thing pipelines eat.
98
99 Roll your own: three lambdas, each returning the next -- every layer is a closure capturing one more
100 argument until the innermost body finally has both. No state, no storage: the captured environment IS the
101 data structure.
102
103 uncurry -- the reverse adaptation: a two-argument function becomes a one-argument function of a PAIR.
104 Reach for it when your data is already (a, b) tuples and your function is not.
105 >>> uncurry(add)((1, 2))
106 3
107 >>> map_(uncurry(lambda a, b: a * b), [(2, 3), (4, 5)])
108 [6, 20]
109
110 partial -- freezes any number of leading arguments, returning a function that takes the rest. Like curry
111 but takes the arguments now, all at once.
112 >>> partial(pow, 2)(10) # pow(2, 10)
113 1024
114 >>> clamp_floor = partial(max, 0) # "never below zero"
115 >>> clamp_floor(-5), clamp_floor(7)
116 (0, 7)
117
118 on -- builds a two-argument function that first sends BOTH arguments through a key function: on(f,
119 key)(x, y) == f(key(x), key(y)). Its natural habitat is comparators: compare two records by one field.
120 >>> on(add, len)("ab", "c") # len("ab") + len("c")
121 3
122 >>> same_length = on(lambda a, b: a == b, len)
123 >>> same_length("abc", "xyz"), same_length("ab", "abc")
124 (True, False)
125 >>> sortBy(on(lambda a, b: a - b, snd), [(1, 9), (2, 3)])
126 [(2, 3), (1, 9)]
127
128 How it works: on(f, key) builds a two-argument function that routes BOTH inputs through key before
129 combining them with f: on(f, key)(x, y) == f(key(x), key(y)). Combination logic and projection logic stay
130 separate, so each is reusable on its own.
131 >>> shorter = on(lambda a, b: a < b, len)
132 >>> shorter("ab", "abc"), shorter("abc", "ab")
133 (True, False)
134
135 Why it earns its place: comparator factories. Any "compare records by one field" collapses to on(compare,
136 field) -- Haskell's sortBy (compare 'on' snd) is exactly sortBy(on(cmp, snd)) here, and reads as its own
137 documentation.
138 >>> sortBy(on(lambda a, b: a - b, snd), [('a', 9), ('b', 1), ('c', 5)])
139 [('b', 1), ('c', 5), ('a', 9)]
140
141 Versus sortOn: when a per-element key exists, sortOn(key, xs) is simpler and faster (the key runs once
142 per element). on shines when an API demands a genuine two-argument comparator, or when the combining
143 function is something other than comparison.
144
145 Remember it as: compare the keys, not the things.
146
147 NOTHING -- a private sentinel object meaning "no argument was supplied". Why not None? Because None is a
148 legitimate VALUE a caller might pass; NOTHING can only mean "omitted", since nobody else has a reference
149 to it. Always test with 'is'. This is how foldl and accumulate tell an absent initial value from a real
150 one.
151 >>> x = NOTHING
152 >>> x is NOTHING
153 True
154 >>> def greet(name=NOTHING):
155 ...     return "hello, stranger" if name is NOTHING else "hello, " + name
156 >>> greet()
157 'hello, stranger'
158 >>> greet("ada")
159 'hello, ada'
160
161 How it works: object() creates a fresh, featureless object whose only property is its IDENTITY -- nothing
162 else in the program can ever be it. Testing x is NOTHING therefore asks exactly one question: "was I
163 handed this precise sentinel?", which makes it a safe flag for "no argument was supplied".
164
165 Why not None, demonstrated: None is data (fromMaybe's whole job is handling it), so using it to also mean
166 "argument omitted" would blur two different situations. The sentinel keeps them apart: None is Nothing in
167 the DATA; NOTHING is Nothing in the ARGUMENT LIST.
168 >>> def describe(x=NOTHING):
169 ...     if x is NOTHING:
170 ...         return "no argument"
171 ...     return "got " + repr(x)
172 >>> describe()
173 'no argument'
174 >>> describe(None)
175 'got None'
176

```

```

177 Where the prelude leans on it: foldl and accumulate take an optional seed. With NOTHING as the default
178 they can tell "seed omitted -- use the first element" from "the seed IS None", which a None default could
179 not.
180
181 Remember it as: None is a value; NOTHING is the absence of one.
182
183 fromMaybe -- lands a default: fromMaybe(d, x) is x unless x is None, in which case it is d. The standard
184 way to finish a pipeline built on the None-is-Nothing convention (find, lookup, getpath, stripPrefix all
185 return None on failure).
186 >>> fromMaybe(0, None), fromMaybe(0, 5)
187 (0, 5)
188 >>> fromMaybe('_', find(lambda c: c == 'q', "abc"))
189 '_'
190 >>> fromMaybe(99, lookup('b', [( 'a', 1), ( 'b', 2)]))
191 2
192
193 Start from a problem you already have: find looks for something that might not be there, and when it is
194 not, you are left holding None -- which you cannot print to a user, add, or put in a report. So you write
195 a backup plan:
196 >>> result = find(even, [3, 5, 7])
197 >>> if result is None:
198 ...     result = 0
199 >>> result
200 0
201
202 fromMaybe is those three lines as one word. Argument order: the backup plan first, then the risky thing
203 -- said aloud it parses: "use 0 if this comes back empty".
204 >>> fromMaybe(0, find(even, [3, 5, 7]))
205 0
206 >>> fromMaybe(0, find(even, [3, 5, 8]))
207 8
208
209 Versus x or d: Python's or falls back on ANYTHING falsy -- 0, "", [], False -- not just None, so it
210 silently destroys legitimate zero-ish answers. fromMaybe asks the only correct question: is this the
211 absence marker?
212 >>> fromMaybe(99, 0), (0 or 99)
213 (0, 99)
214
215 Remember it as: ifNoneUse -- the three-line backup plan as one word; or-with-a-conscience.
216
217 isJust -- the None test as a predicate you can hand to other tools. The lambda it replaces,
218 'lambda v: v is not None', kept reappearing wherever a Maybe-producing step feeds a predicate slot;
219 naming it also locks in the discipline: identity against None, never truthiness, so 0 and '' count as
220 present.
221 >>> isJust(0), isJust(''), isJust(None)
222 (True, True, False)
223 >>> find(isJust, [None, None, 7, None])
224 7
225 >>> all(map(isJust, [1, 0, 'x']))
226 True
227
228 isNothing -- the mirror: True only for None. Counting failures reads as one word.
229 >>> isNothing(None), isNothing(0)
230 (True, False)
231 >>> len(filter_(isNothing, [None, 3, None]))
232 2
233
234 Together they finish the Maybe family: fromMaybe LANDS an optional value, mapMaybe/catMaybes FILTER
235 optional values, isJust/isNothing TEST one. Haskell's Data.Maybe has all five under the same names.
236 >>> readings = [(1, None), (2, 7.5), (3, None)]
237 >>> fromMaybe(-1, find(isJust, map(snd, readings)))
238 7.5
239
240 ===== 2. pairs =====
241
242 fst -- the first element of a pair. Reads better than p[0] in code that is all tuples, and slots into
243 higher-order positions.
244 >>> fst((1, 2))
245 1
246 >>> map_(fst, [(1, 'a'), (2, 'b')])
247 [1, 2]
248
249 snd -- the second element of a pair. The classic sort key for (key, count) items.
250 >>> snd((1, 2))
251 2
252 >>> sortOn(snd, [( 'a', 3), ( 'b', 1)])
253 [( 'b', 1), ( 'a', 3)]
254
255 swap -- exchanges a pair's elements. One honest use: inverting a dict by swapping its items.
256 >>> swap((1, 2))
257 (2, 1)
258 >>> dict(map_(swap, [( 'a', 1), ( 'b', 2)]))
259 {1: 'a', 2: 'b'}
260
261 ===== 3. arithmetic & logic =====
262
263 succ -- successor: one more; on a character (Haskell's Char instance of Enum), the next codepoint.
264 >>> succ(41)

```

```

265 42
266 >>> succ('a')
267 'b'
268 >>> map_(succ, [1, 2, 3])
269 [2, 3, 4]
270
271 pred -- predecessor: one less; characters step back one codepoint.
272 >>> pred(43)
273 42
274 >>> pred('b')
275 'a'
276
277 even -- is n divisible by two? A predicate shaped for filter_ and friends.
278 >>> even(4), even(7)
279 (True, False)
280 >>> filter_(even, [1, 2, 3, 4])
281 [2, 4]
282
283 odd -- the complement of even.
284 >>> odd(4), odd(7)
285 (False, True)
286 >>> partition(odd, [1, 2, 3, 4])
287 ([1, 3], [2, 4])
288
289 not_ -- logical negation as a function ('not' is a keyword, so it cannot be passed around; not_ can).
290 Composes predicates.
291 >>> not_(True)
292 False
293 >>> is_odd = compose(not_, even)
294 >>> is_odd(3)
295 True
296
297 otherwise -- just True, wearing Haskell's name for the catch-all guard. Its habitat is the RULES
298 TABLE: a long if/elif ladder rewritten as data -- (predicate, result) pairs tried in order, find
299 picking the first match. One care: Haskell guards are EVALUATED, so bare otherwise works there; a
300 table's guards are APPLIED to the argument, so wrap it as const(otherwise) -- the guard that ignores
301 its input and says yes. Result slots follow the same rule: fixed answers wear const, computed ones
302 (like str below) go bare.
303 >>> otherwise
304 True
305 >>> RULES = [(lambda n: n % 15 == 0, const("FizzBuzz")),
306 ...         (lambda n: n % 3 == 0, const("Fizz")),
307 ...         (lambda n: n % 5 == 0, const("Buzz")),
308 ...         (const(otherwise), str)]
309 >>> speak = lambda n: snd(find(lambda rule: fst(rule)(n), RULES))(n)
310 >>> map_(speak, [1, 3, 5, 15])
311 ['1', 'Fizz', 'Buzz', 'FizzBuzz']
312
313 signum -- the sign of a number as -1, 0 or 1, computed by arithmetic on two comparisons -- no branches.
314 Also a ready-made comparator result.
315 >>> signum(-7), signum(0), signum(3)
316 (-1, 0, 1)
317 >>> sortBy(lambda a, b: signum(a - b), [3, 1, 2])
318 [1, 2, 3]
319
320 div -- integer division that FLOORS (rounds toward negative infinity), like Haskell's div and Python's
321 //. The div/mod pair and the quot/rem pair agree on positives and split on negatives -- interviews live
322 in that gap.
323 >>> div(7, 2), div(-7, 2)
324 (3, -4)
325
326 mod -- the remainder that matches div: its sign follows the DIVISOR, and div(a,b)*b + mod(a,b) == a
327 always holds.
328 >>> mod(7, 2), mod(-7, 2), mod(7, -2)
329 (1, 1, -1)
330
331 quot -- integer division that TRUNCATES (rounds toward zero), like Haskell's quot and C's /. Same as div
332 for positives; differs by one for negatives.
333 >>> quot(7, 2), quot(-7, 2)
334 (3, -3)
335
336 rem -- the remainder that matches quot: its sign follows the DIVIDEND.
337 >>> rem(7, 2), rem(-7, 2), rem(7, -2)
338 (1, -1, 1)
339
340 quotRem -- truncating quotient and remainder at once.
341 >>> quotRem(-7, 2)
342 (-3, -1)
343
344 gcd -- greatest common divisor by Euclid's recursion, always >= 0 like Haskell's. Recursion is safe here:
345 the depth is at most ~90 even for 64-bit inputs (worst case: consecutive Fibonacci).
346 >>> gcd(48, 18)
347 6
348 >>> gcd(-4, 6), gcd(17, 5)
349 (2, 1)
350
351 lcm -- least common multiple via gcd, also always >= 0; defined as 0 when either input is 0.
352 >>> lcm(4, 6)

```

```

353 12
354 >>> lcm(-4, 6), lcm(0, 5)
355 (12, 0)
356
357 halve -- integer division by two, floored; the natural step for iterate and Newton-style halving.
358 >>> halve(7), halve(-7)
359 (3, -4)
360 >>> list(take(5, iterate(halve, 1024)))
361 [1024, 512, 256, 128, 64]
362
363 hypot -- the Euclidean distance sqrt(x^2 + y^2). Note it returns a float.
364 >>> hypot(3, 4)
365 5.0
366
367 isqrt -- floor square root using Newton's method on INTEGERS -- no floats, so no precision loss on huge
368 numbers; a negative input raises rather than lying.
369 >>> isqrt(16), isqrt(17)
370 (4, 4)
371 >>> isqrt(10**18)
372 10000000000
373
374 Roll your own: Newton's iteration on integers -- repeatedly replace x with the average of x and n // x.
375 It converges from above, so loop while the new estimate still shrinks; all arithmetic is //, so no float
376 (and no float rounding lie) ever appears.
377
378 ===== 4. list basics =====
379
380 head -- the first element. A PARTIAL function: on an empty list it raises, so guard with null() or reach
381 for find/fromMaybe when emptiness is possible.
382 >>> head([1, 2, 3])
383 1
384 >>> head([])
385 Traceback (most recent call last):
386 ...
387 IndexError: list index out of range
388
389 tail -- everything after the first element. head and tail together are the grammar of structural
390 recursion: do something with head, recurse on tail.
391 >>> tail([1, 2, 3])
392 [2, 3]
393 >>> tail([1])
394 []
395
396 init -- everything before the last element (the mirror of tail).
397 >>> init([1, 2, 3])
398 [1, 2]
399
400 last -- the final element. Partial, like head.
401 >>> last([1, 2, 3])
402 3
403
404 null -- is the list empty? The guard that makes head/tail safe to use.
405 >>> null([]), null([1])
406 (True, False)
407
408 elem -- membership test, argument order (needle, haystack).
409 >>> elem(2, [1, 2, 3])
410 True
411 >>> elem('q', "abc")
412 False
413
414 notElem -- non-membership.
415 >>> notElem(9, [1, 2, 3])
416 True
417
418 replicate -- n copies of one value, as a list.
419 >>> replicate(3, 'x')
420 ['x', 'x', 'x']
421 >>> replicate(0, 'x')
422 []
423
424 drop -- discard the first n elements, keep the rest; finite lists only (it materialises the input), and a
425 negative n drops nothing.
426 >>> drop(2, [1, 2, 3, 4])
427 [3, 4]
428 >>> drop(10, [1, 2]), drop(-3, [1, 2])
429 ([], [1, 2])
430
431 splitAt -- cut a list into (first n, rest) in one call; a negative n splits at the front. The pieces glue
432 back with +, which is why rotations and chunking fall out of it.
433 >>> splitAt(2, [1, 2, 3, 4])
434 ([1, 2], [3, 4])
435 >>> splitAt(0, [1, 2]), splitAt(-2, [1, 2])
436 ([], [1, 2]), ([], [1, 2])
437
438 nub -- deduplicate, FIRST occurrence wins, order preserved. The trick: dict.fromkeys builds a dict (whose
439 keys are unique and insertion-ordered) and throws away the values. Elements must be hashable. Compare
440 set(xs), which destroys order.

```

```

441 >>> nub([3, 1, 3, 2, 1])
442 [3, 1, 2]
443 >>> nub("mississippi")
444 ['m', 'i', 's', 'p']
445
446 lookup -- the first value paired with a key in an association list (a list of (key, value) tuples), or
447 None if absent. First match wins, so an assoc list can shadow like a scope chain.
448 >>> lookup('b', [('a', 1), ('b', 2)])
449 2
450 >>> lookup('a', [('a', 1), ('a', 99)])
451 1
452 >>> lookup('z', [('a', 1)]) is None
453 True
454
455 zip_ -- pair two iterables positionally, stopping at the shorter; returns a real list (the builtin zip
456 returns a lazy iterator), and either side may be any iterable, streams included.
457 >>> zip_([1, 2], [10, 20, 30])
458 [(1, 10), (2, 20)]
459 >>> zip_("ab", count(0))
460 [('a', 0), ('b', 1)]
461
462 zip3 -- the three-list version.
463 >>> zip3([1, 2], ['a', 'b'], [True, False])
464 [(1, 'a', True), (2, 'b', False)]
465
466 unzip -- a list of pairs back into a pair of lists; the inverse of zip_.
467 >>> unzip([(1, 'a'), (2, 'b')])
468 ([1, 2], ['a', 'b'])
469 >>> unzip([])
470 ([], [])
471
472 transpose -- rows become columns, RAGGED-SAFE exactly like Haskell's: short rows simply drop out of later
473 columns; nothing is silently truncated.
474 >>> transpose([[1, 2, 3], [4, 5, 6]])
475 [[1, 4], [2, 5], [3, 6]]
476 >>> transpose([[1, 2, 3], [4, 5]])
477 [[1, 4], [2, 5], [3]]
478 >>> transpose([])
479 []
480
481 Rows may be any iterables, even one-shot ones -- the rows are materialised first, so a generator of
482 iterators transposes like a list of lists (without that step the generator is spent by the length scan):
483 >>> transpose(iter([1, 2]) for _ in range(2))
484 [[1, 1], [2, 2]]
485 >>> transpose(["abc", "de"])
486 [['a', 'd'], ['b', 'e'], ['c']]
487
488 Roll your own: materialise the rows, find the longest, then for each column index keep xs[i] only from
489 rows that reach it -- the 'if i < len(xs)' filter is precisely where short rows drop out of later columns
490 instead of truncating everyone.
491
492 enum -- pair every element with its index: a list-returning enumerate. The shape that feeds index-aware
493 folds (two_sum's seen-dict, next_greater's stack).
494 >>> enum(['a', 'b'])
495 [(0, 'a'), (1, 'b')]
496 >>> enum(['a', 'b'], start=1)
497 [(1, 'a'), (2, 'b')]
498
499 pairwise -- every element with its RIGHT neighbour: zip xs (tail xs). The tool for any rule about
500 adjacency -- deltas, local comparisons, Roman numeral subtraction.
501 >>> pairwise([1, 4, 9])
502 [(1, 4), (4, 9)]
503 >>> map_(lambda p: p[1] - p[0], pairwise([3, 10, 6]))
504 [7, -4]
505
506 isPrefixOf -- does xs start with p? Works on strings and lists alike (both are normalised through
507 list()).
508 >>> isPrefixOf("re", "rebuild")
509 True
510 >>> isPrefixOf([1, 2], [1, 2, 3])
511 True
512 >>> isPrefixOf("", "anything")
513 True
514
515 isSuffixOf -- does xs end with p? Implemented by slicing from len(xs) - len(p), NOT by xs[-len(p):],
516 because when p is empty that slice is xs[-0:] == the WHOLE list. A scar worth keeping visible.
517 >>> isSuffixOf("ing", "folding")
518 True
519 >>> isSuffixOf("", "x")
520 True
521 >>> isSuffixOf("abc", "bc")
522 False
523
524 stripPrefix -- Just the rest after the prefix, or Nothing: returns xs[len(p):] when the prefix matches,
525 None when it does not. Pairs with fromMaybe and mapMaybe.
526 >>> stripPrefix("re", "rebuild")
527 'build'
528 >>> stripPrefix("un", "rebuild") is None

```

```

529 True
530 >>> mapMaybe(partial(stripPrefix, "err:"), ["err:a", "ok:b", "err:c"])
531 ['a', 'c']
532
533 ===== 5. higher-order lists =====
534
535 map_ -- apply a function to every element; returns a real list, eagerly (the builtin map is lazy). The
536 workhorse transformation.
537 >>> map_(lambda x: x * x, [1, 2, 3])
538 [1, 4, 9]
539 >>> map_(str.upper, ["ab", "cd"])
540 ['AB', 'CD']
541
542 filter_ -- keep the elements satisfying a predicate. Scans the WHOLE list; if you only want the first
543 hit, that is find.
544 >>> filter_(even, [1, 2, 3, 4])
545 [2, 4]
546 >>> filter_(str.isdigit, "alb22")
547 ['1', '2', '2']
548
549 find -- the first element satisfying the predicate, else None. Lazy: it stops at the first hit, which
550 folds cannot do, and therefore works even on infinite streams. Finish with fromMaybe if you need a
551 default.
552 >>> find(even, [1, 3, 4, 6])
553 4
554 >>> find(even, [1, 3]) is None
555 True
556 >>> find(lambda x: x % 7 == 0, count(1))
557 7
558
559 How it works: find wraps a generator expression in next() with a None default. The generator is the point
560 -- elements are tested one at a time, on demand, and the search STOPS at the first hit. Nothing after the
561 match is ever examined.
562
563 Why it earns its place: folds and filter_ consume the whole list; they cannot stop early. find can --
564 which makes it the right tool the moment the word "first" appears in a statement, and the only tool that
565 survives an infinite stream. Land defaults with fromMaybe.
566 >>> find(lambda x: x * x > 50, count(1))
567 8
568 >>> fromMaybe('?', find(str.isdigit, "abc7de"))
569 '7'
570
571 Versus filter_: filter_ answers "which ones?"; find answers "who was first, if anyone?". Taking
572 filter_(p, xs)[0] instead does wasted work and explodes when nothing matches; find does neither.
573
574 Remember it as: find is the early exit that folds cannot perform.
575
576 Roll your own: a generator expression handed to next() with a default. The generator is what makes it
577 lazy -- elements are produced one at a time, so the first hit stops everything -- and the default is what
578 makes it total.
579
580 partition -- split into (satisfy, don't) in ONE pass, both sides in original order; the predicate runs
581 exactly once per element, so it may be expensive or even stateful.
582 >>> partition(even, [1, 2, 3, 4])
583 ([2, 4], [1, 3])
584 >>> partition(str.isalpha, "alb2")
585 (['a', 'b'], ['1', '2'])
586
587 Roll your own: two buckets and one conditional append -- (yes if crit(x) else no).append(x). The
588 conditional expression picks the LIST, then the append mutates it; one pass, exactly one predicate call
589 per element.
590
591 mapMaybe -- map a function that returns a value or None, and keep only the values: parse-and-filter in
592 ONE pass. The shape for messy input -- write a parser that answers None for garbage, then mapMaybe it
593 over everything. dict.get is already such a function.
594 >>> mapMaybe(lambda s: int(s) if s.isdigit() else None, ["3", "x", "7"])
595 [3, 7]
596 >>> mapMaybe({'a': 1, 'b': 2}.get, ['a', 'x', 'b'])
597 [1, 2]
598
599 How it works: run f over every element; f answers a VALUE when it can make sense of the input and None
600 when it cannot; keep only the values. Map and filter fused into one pass, with the parser itself deciding
601 what survives.
602 >>> parse_int = lambda s: int(s) if s.lstrip('-').isdigit() else None
603 >>> mapMaybe(parse_int, ["12", "x", "-3", ""])
604 [12, -3]
605
606 Why it earns its place: messy input. The alternative is a loop with an if, a flag and an append -- three
607 chances for a bug. Here the contract is crisp: write parse(item) -> value-or-None once, and mapMaybe
608 handles every item. dict.get is already such a function, so lookups that may miss compose directly.
609 >>> legend = {'a': 1, 'b': 2}
610 >>> mapMaybe(legend.get, "cabbage")
611 [1, 2, 2, 1]
612
613 The Haskell connection: Data.Maybe's mapMaybe, with None standing in for Nothing -- the "parse, don't
614 validate" discipline: turn questionable data into clean data at the boundary, and everything downstream
615 stops worrying.
616

```

```

617 Remember it as: map with a parser, drop the Nothings, one pass.
618
619 catMaybes -- keep the non-None values; mapMaybe with the identity.
620 >>> catMaybes([1, None, 2, None])
621 [1, 2]
622
623 concat -- flatten ONE level of nesting.
624 >>> concat([1, 2], [3], [])
625 [1, 2, 3]
626 >>> concat(["ab", "cd"])
627 ['a', 'b', 'c', 'd']
628
629 concatMap -- map each element to a LIST, and pool all the results: concat after map_, fused. The backbone
630 of enumeration -- "for every choice, produce all outcomes".
631 >>> concatMap(lambda w: [w, w.upper()], ["a", "b"])
632 ['a', 'A', 'b', 'B']
633 >>> concatMap(lambda x: [x * x, [1, 2, 3]])
634 [1, 2, 2, 3, 3, 3]
635 >>> concatMap(lambda x: [x, -x], [1, 2]) == concat(map_(lambda x: [x, -x], [1, 2]))
636 True
637
638 starMap -- map a MULTI-argument function over a list of argument tuples: map_ composed with uncurry.
639 >>> starMap(lambda a, b: a * b, [(2, 3), (4, 5)])
640 [6, 20]
641 >>> starMap(pow, [(2, 3), (3, 2)])
642 [8, 9]
643
644 zipWith -- combine two lists elementwise with a function; stops at the shorter. zip_ is just
645 zipWith(make-a-pair).
646 >>> zipWith(add, [1, 2], [10, 20])
647 [11, 22]
648 >>> zipWith(lambda a, b: a > b, [3, 1], [2, 5, 9])
649 [True, False]
650
651 zipWith3 -- the three-list version.
652 >>> zipWith3(lambda a, b, c: a + b + c, [1], [2], [3])
653 [6]
654
655 cross -- every pairing of two lists (the cartesian product). For the product of a list with ITSELF n
656 times, see replicateM.
657 >>> cross([1, 2], "ab")
658 [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
659 >>> len(cross(range(3), range(4)))
660 12
661
662 ===== 6. folds =====
663
664 foldl -- THE left fold, and the prelude's most important definition. It is exactly this loop: acc = base;
665 for x in xs: acc = f(acc, x); return acc. The combiner takes (ACCUMULATOR, element) -- accumulator first.
666 Argument order of foldl itself: (f, xs, base), with base optional -- omitted, the first element seeds the
667 accumulator (and an empty list is then an error).
668 >>> foldl(lambda acc, x: acc * 10 + x, [1, 2, 3], 0)
669 123
670 >>> foldl(max, [3, 9, 2]) # no base: 3 seeds it
671 9
672 >>> foldl(add, [], 0) # empty + base: the base comes back
673 0
674 >>> foldl(lambda acc, w: acc + len(w), ["ab", "c"], 0)
675 3
676
677 How it works: foldl is precisely this loop, given a name and a contract:
678 >>> def foldl_spelled_out(f, xs, base):
679 ...     acc = base
680 ...     for x in xs:
681 ...         acc = f(acc, x)
682 ...     return acc
683 >>> foldl_spelled_out(add, [1, 2, 3], 0) == foldl(add, [1, 2, 3], 0)
684 True
685
686 Two disciplines hide in the signature. The combiner takes (ACCUMULATOR, element) -- accumulator first,
687 always; reversing them is the classic fold bug. And the optional base is guarded by NOTHING, so the fold
688 can tell "no seed -- use the first element" from "the seed is None".
689
690 Why it earns its place: the fold is the universal list consumer. sum, max, reverse, "build a dict", "run
691 a state machine over events" -- each is one choice of combiner and seed. When you can say "carry THIS
692 state, update it THIS way per element", the loop is already written.
693 >>> foldl(lambda acc, x: [x] + acc, [1, 2, 3], []) # reverse is a fold
694 [3, 2, 1]
695 >>> foldl(lambda acc, w: acc + len(w), ["ab", "cde"], 0)
696 5
697
698 Remember it as: a fold is a for-loop whose state has been given a contract.
699
700 Roll your own: seed the accumulator -- base if one was given, else the first element (the NOTHING guard
701 is what lets None be a legitimate seed) -- then a single for-loop folding acc = f(acc, x). The entire
702 contract is which side the accumulator sits on.
703
704 foldl1 -- foldl seeded by the first element, spelled out. Use when the accumulator has the same type as

```



```

705 the elements and emptiness cannot happen.
706 >>> foldl1(add, [1, 2, 3])
707 6
708 >>> foldl1(min, [3, 1, 2])
709 1
710
711 foldr -- the RIGHT fold: nests from the right, f takes (element, ACCUMULATOR) -- element first, mirror of
712 foldl. Implemented as foldl of the flipped function over the reversed list: iterative and stack-safe, an
713 identity that only holds because Python is strict. The parenthesis example makes the direction
714 visible.
715 >>> foldr(lambda x, acc: '(' + x + '+' + acc + ')', "abc", "z")
716 '(a+(b+(c+z)))'
717 >>> foldl(lambda acc, x: '(' + acc + '+' + x + ')', "abc", "z")
718 '(((z+a)+b)+c)'
719 >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
720 [1, 2, 3]
721
722 How it works: the mirror image -- the combiner takes (element, ACCUMULATOR) and the nesting grows from
723 the RIGHT: f(a, f(b, f(c, z))). The prelude does not write a second recursion: foldr = foldl(flip(f),
724 reversed(xs), base). Flip fixes the argument order, reversing fixes the traversal, and the existing
725 engine runs backwards.
726
727 The honesty clause: that identity only holds because Python is STRICT. Haskell's foldr can process
728 infinite lists -- laziness lets f decide whether the rest is ever needed; ours materialises and reverses
729 the list first, so it cannot. Same name, same finite behaviour, different superpower.
730
731 Why it earns its place: building right-to-left. Consing onto a front is natural from the right -- section
732 13's from_list IS a foldr of ListNode, and here it is, derived by hand:
733 >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], []) # rebuild, order kept
734 [1, 2, 3]
735 >>> to_list(foldr(ListNode, [1, 2, 3], None)) # from_list, spelled out
736 [1, 2, 3]
737
738 Remember it as: foldl eats left-to-right; foldr thinks right-to-left -- and in strict Python it is foldl
739 wearing flip and a reversal.
740
741 foldr1 -- right fold seeded by the LAST element.
742 >>> foldr1(lambda x, acc: x - acc, [1, 2, 3]) # 1 - (2 - 3)
743 2
744
745 product -- multiply everything: the numeric fold, seeded with 1 so an empty product is 1.
746 >>> product([2, 3, 4])
747 24
748 >>> product([])
749 1
750
751 compose -- fold any number of functions into one; the RIGHTMOST runs first, like mathematical composition
752 f(g(x)).
753 >>> compose(str.strip, str.upper)(" hi ")
754 'HI'
755 >>> f = compose(succ, succ, succ)
756 >>> f(0)
757 3
758
759 How it works: a fold over the functions themselves -- neighbours merge into lambda x: f(g(x)), seeded
760 with the identity, so compose() with no arguments is id. The RIGHTMOST function runs first, matching
761 mathematical (f . g)(x) = f(g(x)).
762 >>> compose()(42) == id(42)
763 True
764 >>> compose(len, str.strip)(" hi ") # strip first, then len
765 2
766
767 Why it earns its place: it turns a pipeline into a VALUE. A composed function can be named, mapped,
768 stored in a rules table -- processing steps become data you can shuffle. The constraint is the one curry
769 answers: every stage must be unary, which is why compose-heavy code leans on curry and partial to shape
770 its stages.
771 >>> normalize = compose(str.lower, str.strip)
772 >>> map_(normalize, [" Hi ", " YO "])
773 ['hi', 'yo']
774
775 Remember it as: foldr (.) id -- a pipeline you can hold in your hand.
776
777 Roll your own: it IS foldr (.) id -- foldr over the functions with x as the seed, each stage applied to
778 the accumulated result: foldr(lambda f, acc: f(acc), fns, x). Because the prelude's foldr is an iterative
779 loop underneath, stack depth stays constant no matter how many stages you compose.
780
781 subsequences -- every subset of the elements (the powerset), built by a fold in which each element
782 DOUBLES the accumulator: keep every existing subset, and every existing subset extended by the newcomer.
783 2^n results -- a brute-force license for n up to about 20, and you should say that bound out loud before
784 using it.
785 >>> subsequences([1, 2])
786 [[], [1], [2], [1, 2]]
787 >>> subsequences("ab")
788 [[], ['a'], ['b'], ['a', 'b']]
789 >>> len(subsequences("abcd"))
790 16
791
792 How it works: watch the accumulator double. Start from [[]] -- one subset, the empty one. Each element x

```

```

793 rewrites acc as acc + [s + [x] for s in acc]: every existing subset survives (x left out), and every
794 existing subset reappears extended by x (x taken). n elements, n doublings, 2^n subsets, in a stable
795 order.
796 >>> acc = [[]]
797 >>> acc = acc + [s + [1] for s in acc]; acc
798 [[], [1]]
799 >>> acc = acc + [s + [2] for s in acc]; acc
800 [[], [1], [2], [1, 2]]
801
802 Why it earns its place: it is a brute-force LICENSE. At n <= 20, checking every subset is a few million
803 cheap operations -- honest, exhaustive, and unbeatable to debug. Say the bound out loud ("2^15 is 32768
804 -- fine") and enumerate; cleverness can come later if the bound grows.
805 >>> best = maxOn(sum, [s for s in subsequences([3, -1, 4, -2]) if sum(s) <= 6])
806 >>> sum(best)
807 6
808
809 Remember it as: the powerset fold -- each element doubles the world.
810
811 replicateM -- every length-n word over an alphabet: cross applied n times, |xs|^n results. The other
812 brute-force license -- all bitstrings, all dice rolls, all assignments of n slots.
813 >>> replicateM(2, "ab")
814 [['a', 'a'], ['a', 'b'], ['b', 'a'], ['b', 'b']]
815 >>> len(replicateM(3, [0, 1]))
816 8
817
818 ===== 7. scans =====
819
820 accumulate -- the generator behind the scans: yields the running combination of everything seen so far.
821 With no function it sums; 'initial' seeds it. This is Python's itertools name with Haskell's scan
822 semantics.
823 >>> list(accumulate([1, 2, 3]))
824 [1, 3, 6]
825 >>> list(accumulate([1, 2], initial=10))
826 [10, 11, 13]
827 >>> list(accumulate([3, 1, 2], min))
828 [3, 1, 1]
829
830 Roll your own: the same skeleton as foldl, but a GENERATOR -- yield the accumulator once before the loop,
831 then yield again after every combine. The early bare return on an empty, seedless input is what makes an
832 empty scan come back empty instead of raising.
833
834 scanl -- a fold that KEEPS ITS HISTORY: returns every intermediate accumulator, starting with the seed,
835 so the result has len(xs)+1 elements and its last element equals the foldl. Argument order (f, z, xs) --
836 seed in the middle, Haskell's order, unlike foldl's trailing base. Prefix sums are the canonical scan.
837 >>> scanl(add, 0, [3, 1, 4])
838 [0, 3, 4, 8]
839 >>> last(scanl(add, 0, [1, 2, 3])) == foldl(add, [1, 2, 3], 0)
840 True
841
842 The DP secret: many celebrated one-pass algorithms are scans in a trench coat, because a scan IS a
843 dynamic-programming table -- entry i holds the answer for the prefix ending at i. Kadane's famous
844 maximum-subarray "trick" is one scanl and a max:
845 >>> xs = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
846 >>> max(scanl(lambda best, x: max(x, best + x), xs))
847 6
848
849 Prefix sums, the other superpower: scan once with (+) and every range sum becomes two reads, pref[j] -
850 pref[i] -- O(n) window questions collapse to O(1).
851 >>> pref = scanl(add, 0, [3, 1, 4, 1, 5])
852 >>> pref[4] - pref[1] # sum of elements 1..3
853 6
854
855 Remember it as: when you need the state at EVERY step, one scan replaces n folds.
856
857 scanl1 -- the history-keeping fold seeded by the first element; len(xs) results. Running maxima and
858 minima live here.
859 >>> scanl1(min, [3, 1, 2])
860 [3, 1, 1]
861 >>> scanl1(max, [1, 3, 2, 5])
862 [1, 3, 3, 5]
863
864 scanr -- the suffix scan: folds from the RIGHT, showing every suffix's result; the first element is the
865 full fold, the last is the seed. Suffix sums for product_except_self-style problems.
866 >>> scanr(add, 0, [1, 2, 3])
867 [6, 5, 3, 0]
868
869 scanr1 -- suffix scan seeded by the last element.
870 >>> scanr1(add, [1, 2, 3])
871 [6, 5, 3]
872
873 ===== 8. streams =====
874
875 count -- the infinite arithmetic sequence start, start+step, ... Never consume it whole: pair it with
876 take, takewhile or find, which stop.
877 >>> take(3, count(10))
878 [10, 11, 12]
879 >>> take(3, count(0, 5))
880 [0, 5, 10]

```

```

881
882 repeat -- one value forever, or exactly n times with the second argument.
883 >>> take(2, repeat('x'))
884 ['x', 'x']
885 >>> list(repeat('x', 3))
886 ['x', 'x', 'x']
887 >>> list(repeat('x', 0))
888 []
889
890 cycle -- a finite list looped forever; an EMPTY list raises immediately (Haskell errors too -- the
891 alternative is a silent infinite hang).
892 >>> take(5, cycle([1, 2]))
893 [1, 2, 1, 2, 1]
894
895 iterate -- x, f(x), f(f(x)), ... forever: repeated application as a stream. Any deterministic state
896 machine becomes iterate on its step function -- Fibonacci is iterate on a pair.
897 >>> take(4, iterate(lambda x: 2 * x, 1))
898 [1, 2, 4, 8]
899 >>> step = lambda p: (snd(p), fst(p) + snd(p))
900 >>> map_(fst, take(6, iterate(step, (0, 1))))
901 [0, 1, 1, 2, 3, 5]
902
903 unfoldr -- iterate's FINITE twin, and the cure for threading (state, acc) tuples through loops. You
904 supply f(seed) returning either (value, seed') to emit a value and continue, or None to stop; unfoldr
905 collects the values. Anything that "peels" a structure -- digits, chunks, parent chains -- is an unfoldr.
906 >>> unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), 407)
907 [7, 0, 4]
908 >>> unfoldr(lambda n: None if n == 0 else (n, n - 1), 5)
909 [5, 4, 3, 2, 1]
910 >>> unfoldr(const(None), "seed")          # stop immediately: empty
911 []
912
913 How it works: the step function carries a two-outcome contract -- f(seed) returns (value, next_seed) to
914 emit and continue, or None to stop -- and unfoldr collects what was emitted. It is the fold REVERSED: a
915 fold consumes a list into a summary; an unfold grows a list out of a seed. (Category theory says
916 anamorphism; interviewers say "generate the sequence".)
917
918 Why it earns its place: it retires the (state, accumulator) tuple you would otherwise thread through a
919 while loop. Anything that PEELS -- digits off a number, chunks off a list, parents up a tree, a Collatz
920 trajectory -- states its entire logic in the step function.
921 >>> unfoldr(lambda n: None if n == 1 else (n, n // 2 if even(n) else 3 * n + 1), 6)
922 [6, 3, 10, 5, 16, 8, 4, 2]
923
924 Versus iterate: iterate is the infinite cousin -- an unfold whose step never says None, consumed with
925 take. Reach for iterate when the stream is conceptually endless, unfoldr when the seed itself knows where
926 the end is.
927
928 Remember it as: fold tears down; unfold builds up; the seed carries the future.
929
930 More unfolds, same two-outcome contract -- peeling chunks off a list, bits off a number, walking a parent
931 chain to its root:
932 >>> unfoldr(lambda xs: None if xs == [] else (xs[:2], xs[2:]), [1, 2, 3, 4, 5])
933 [[1, 2], [3, 4], [5]]
934 >>> unfoldr(lambda m: None if m == 0 else (m % 2, m // 2), 13)          # bits, LSB first
935 [1, 0, 1, 1]
936 >>> parents = {'c': 'b', 'b': 'a', 'a': None}
937 >>> unfoldr(lambda n: None if n is None else (n, parents[n]), 'c')      # ancestry
938 ['c', 'b', 'a']
939
940 And the seed can carry several facts at once -- here (current, next, remaining) generates Fibonacci with
941 its own countdown built in:
942 >>> unfoldr(lambda s: None if s[2] == 0 else (s[0], (s[1], s[0] + s[1], s[2] - 1)), (0, 1, 8))
943 [0, 1, 1, 2, 3, 5, 8, 13]
944
945 Roll your own: prime the pump with step = f(seed), then loop while step is not None -- append the value,
946 feed the new seed back to f. The two-outcome contract lives entirely in f; unfoldr is just the loop that
947 honours it, plus the list that collects what was emitted.
948
949 chain -- concatenate iterables lazily, one after another: a generator expression, so the tail is never
950 touched until asked for (chain over count() is fine). concat is the strict, list-building cousin.
951 >>> list(chain([1], [2, 3]))
952 [1, 2, 3]
953 >>> take(4, chain([1, 2], count(10)))
954 [1, 2, 10, 11]
955
956 ===== 9. slicing & spans =====
957
958 islice -- at most the first n items of ANY iterable, lazily; the safe window onto an infinite stream.
959 Runs short without complaint.
960 >>> list(islice(count(0), 3))
961 [0, 1, 2]
962 >>> list(islice([1, 2], 5))
963 [1, 2]
964
965 take -- the first n, as a real list. take/drop are the positional pair; takeWhile/dropWhile are the
966 conditional pair.
967 >>> take(3, count(1))
968 [1, 2, 3]

```

```

969 >>> take(2, [7, 8, 9])
970 [7, 8]
971
972 takewhile -- yield elements WHILE the predicate holds, then stop at the first failure (a generator).
973 Contrast filter_: filter_ scans everything; takewhile stops -- on [2, 4, 5, 6], filter_ keeps the 6,
974 takewhile does not.
975 >>> list(takewhile(even, [2, 4, 5, 6]))
976 [2, 4]
977 >>> filter_(even, [2, 4, 5, 6])
978 [2, 4, 6]
979
980 takeuntil -- like takewhile(not p), but INCLUDES the first element where p holds -- the do-while flavour:
981 "read up to and including the terminator".
982 >>> list(takeuntil(lambda x: x == 3, [1, 2, 3, 4]))
983 [1, 2, 3]
984 >>> list(takeuntil(lambda x: x > 0, [5, 6]))
985 [5]
986
987 dropwhile -- discard while the predicate holds, then yield EVERYTHING after the first failure, no more
988 testing. takewhile's complement.
989 >>> list(dropwhile(lambda x: x < 3, [1, 2, 3, 1]))
990 [3, 1]
991
992 takeWhile -- takewhile, delivered as a list (capital W = list version).
993 >>> takeWhile(lambda x: x < 3, [1, 2, 3, 1])
994 [1, 2]
995
996 dropWhile -- dropwhile, delivered as a list.
997 >>> dropWhile(lambda x: x < 3, [1, 2, 3, 1])
998 [3, 1]
999
1000 span -- split at the first failure: (longest satisfying prefix, the rest), computed in ONE pass so it is
1001 safe on one-shot iterators (the old two-scan version silently broke on generators -- this is the fix). A
1002 lexer in one call.
1003 >>> span(lambda x: x < 3, [1, 2, 3, 4, 1])
1004 ([1, 2], [3, 4, 1])
1005 >>> span(str.isdigit, "123abc")
1006 (['1', '2', '3'], ['a', 'b', 'c'])
1007
1008 Roll your own: materialise xs once (that is the one-pass guarantee), locate the first failing index with
1009 next() over enumerate -- defaulting to len(xs) when the predicate never fails -- and slice twice. The
1010 previous two-scan version ran takewhile and dropwhile separately and silently broke on one-shot
1011 generators; this is the repair.
1012
1013 break_ -- span with the predicate negated: split where the condition first HOLDS.
1014 >>> break_(lambda x: x > 2, [1, 2, 3, 1])
1015 ([1, 2], [3, 1])
1016
1017 inits -- every prefix, from empty to the whole thing; len(xs)+1 of them.
1018 >>> inits([1, 2])
1019 [[], [1], [1, 2]]
1020 >>> inits("ab")
1021 ['', 'a', 'ab']
1022
1023 tails -- every suffix, whole thing first, down to empty. Substring problems start here: every substring
1024 is a prefix of some suffix.
1025 >>> tails("abc")
1026 ['abc', 'bc', 'c', '']
1027 >>> tails([1, 2])
1028 [[1, 2], [2], []]
1029
1030 chunksOf -- consecutive pieces of length n; the last may be short. Works on lists and strings alike since
1031 it is pure slicing.
1032 >>> chunksOf(2, [1, 2, 3, 4, 5])
1033 [[1, 2], [3, 4], [5]]
1034 >>> chunksOf(3, "abcdefg")
1035 ['abc', 'def', 'g']
1036
1037 ===== 10. sorting & searching =====
1038
1039 sortOn -- sort by a key function, evaluated once per element. TUPLE KEYS are the power move: (primary,
1040 secondary) sorts two levels at once, and negating a numeric component flips just that level -- (-count,
1041 word) is "count descending, ties alphabetical", which reverse=True cannot express.
1042 >>> sortOn(len, ["bbb", "a", "cc"])
1043 ['a', 'cc', 'bbb']
1044 >>> sortOn(lambda kv: (-kv[1], kv[0]), [('b', 2), ('a', 2), ('c', 3)])
1045 [('c', 3), ('a', 2), ('b', 2)]
1046
1047 minOn -- the element with the smallest key, in O(n). sortOn + head gives the same answer for O(n log n)
1048 -- pay the sort only when you need ALL of them ordered.
1049 >>> minOn(abs, [-3, 2, 5])
1050 2
1051 >>> minOn(snd, [('a', 9), ('b', 1)])
1052 ('b', 1)
1053
1054 maxOn -- the element with the largest key, O(n); the FIRST winner on ties, so pre-sorting the input
1055 controls the tie-break.
1056 >>> maxOn(len, ["aa", "b", "cc"])

```

```

1057 'aa'
1058
1059 cmp_to_key -- adapts an old-style comparator (returning negative, zero or positive) into the key=
1060 protocol. Reach for it only when a rule genuinely compares two elements against each other and no per-
1061 element key exists.
1062 >>> sorted(["bb", "a"], key=cmp_to_key(lambda a, b: len(a) - len(b)))
1063 ['a', 'bb']
1064
1065 Roll your own: a tiny wrapper class implementing ONE dunder -- __lt__ defers to cmp(a, b) < 0. Python's
1066 sort only ever asks "is a less than b?", so that single method is enough to smuggle any comparator
1067 through the key= interface.
1068
1069 sortBy -- comparator sort, Python 2 style, built on cmp_to_key.
1070 >>> sortBy(lambda a, b: b - a, [1, 3, 2])
1071 [3, 2, 1]
1072
1073 merge -- join two ALREADY-SORTED lists into one sorted list, stably, in O(n+m): the two-pointer loop at
1074 the heart of merge sort.
1075 >>> merge([1, 4, 6], [2, 3, 7])
1076 [1, 2, 3, 4, 6, 7]
1077 >>> merge([], [1, 2])
1078 [1, 2]
1079
1080 Roll your own: two cursors, always copying the smaller head forward; the <= (rather than <) is what makes
1081 the merge STABLE, preferring the left list on ties. When either side runs dry, glue both tails on -- one
1082 of them is empty, so the + is harmless.
1083
1084 bisect_left -- binary search over a SORTED list: the first index whose element is >= x; equivalently,
1085 where x would insert to the left of its equals. If a[i] != x there, x is absent -- that check is binary
1086 search.
1087 >>> bisect_left([10, 20, 20, 30], 20)
1088 1
1089 >>> bisect_left([10, 20, 30], 25)          # insertion point for a missing value
1090 2
1091
1092 How it works: on a sorted list the predicate "element < x" reads True for a prefix and False for the rest
1093 -- TTTFFFF. Binary search does not hunt for x; it finds that BOUNDARY, halving the interval per probe.
1094 bisect_left returns the first index >= x, bisect_right the first > x, and from those two facts three
1095 idioms fall out.
1096 >>> xs = [10, 20, 20, 30]
1097 >>> bisect_left(xs, 20), bisect_right(xs, 20)      # the run of 20s occupies [1, 3)
1098 (1, 3)
1099
1100 Idiom one, membership: x is present iff the slot bisect_left names actually holds it. Idiom two,
1101 insertion point: that same index is where x belongs to keep the list sorted. Idiom three, counting: right
1102 minus left counts occurrences -- and with two different probes, the values inside any range.
1103 >>> i = bisect_left(xs, 25)
1104 >>> i, (i < len(xs) and xs[i] == 25)              # 25 would insert at 3; absent
1105 (3, False)
1106
1107 Why it earns its place: every threshold ladder is secretly a sorted list. Grading bands, tax brackets,
1108 rate tiers -- one bisect replaces the if/elif staircase:
1109 >>> grade = lambda score: "FDCBA"[bisect_right([60, 70, 80, 90], score)]
1110 >>> grade(59), grade(60), grade(95)
1111 ('F', 'D', 'A')
1112
1113 Remember it as: bisect finds the boundary in a sea of Trues and Falses; everything else is reading that
1114 boundary three ways.
1115
1116 Roll your own: the lo/hi halving loop over an invariant -- everything left of lo is < x, everything from
1117 hi rightwards is >= x. One comparison, a[mid] < x, decides which half survives; when lo meets hi, that
1118 meeting point IS the boundary. bisect_right is the identical loop with <= in place of < -- one character
1119 moves the boundary to the far side of the equals.
1120
1121 bisect_right -- the first index whose element is > x: where x would insert to the RIGHT of its equals.
1122 right minus left counts the occurrences.
1123 >>> bisect_right([10, 20, 20, 30], 20)
1124 3
1125 >>> xs = [10, 20, 20, 30]
1126 >>> bisect_right(xs, 20) - bisect_left(xs, 20)
1127 2
1128
1129 ===== 11. strings =====
1130
1131 words -- split on any whitespace; runs collapse, edges vanish. The inverse direction is unwords, and the
1132 round trip normalises spacing.
1133 >>> words("  such  spacing ")
1134 ['such', 'spacing']
1135 >>> unwords(words("  a  b "))
1136 'a b'
1137
1138 unwords -- join words with single spaces.
1139 >>> unwords(['a', 'b', 'c'])
1140 'a b c'
1141
1142 lines -- split a string into its lines (no trailing newlines kept).
1143 >>> lines("one\ntwo")
1144 ['one', 'two']

```

```

1145 unlines -- join lines, TERMINATING each with a newline (not separating -- note the trailing one).
1146 >>> unlines(['a', 'b'])
1147 'a\nb\n'
1148
1149
1150 ===== 12. containers =====
1151
1152 defaultdict -- a dict whose missing keys create themselves via a factory: touch d[k] and factory() is
1153 installed there. This "autovivification" makes grouping and counting one-liners. Gotcha: READING a
1154 missing key also plants it -- use 'in' or .get when you only want to look.
1155 >>> d = defaultdict(list)
1156 >>> d['x'].append(1)
1157 >>> d
1158 {'x': [1]}
1159 >>> d2 = defaultdict(int)
1160 >>> d2['ghost'] # a mere read...
1161 0
1162 >>> 'ghost' in d2 # ...planted the key
1163 True
1164
1165 How it works: one dunder does everything. When d[k] misses, Python calls d.__missing__(k); this subclass
1166 responds by calling the stored factory, installing the result, and returning it. The factory runs only on
1167 a miss -- and it takes NO arguments, which is why the leaf recipes in Tree/ITree are zero-argument
1168 lambdas.
1169
1170 Why it earns its place: it deletes the check-then-create dance from every grouping and counting loop. The
1171 write path becomes a bare append or +=, and the structure assembles itself on touch -- autovivification,
1172 which section 12's trie tools then apply recursively.
1173
1174 The discipline it demands: a mere READ also plants the key, so split your paths -- write with d[k], but
1175 ask questions with 'in' or .get. The prelude's bag operations read Counters with .get for exactly this
1176 reason.
1177 >>> d = defaultdict(int)
1178 >>> d['hits'] += 1 # write path: indexing is the point
1179 >>> d.get('misses', 0), 'misses' in d # read path: no ghost key planted
1180 (0, False)
1181
1182 Remember it as: index to build, .get to ask.
1183
1184 Roll your own: subclass dict and implement one dunder -- __missing__, which Python calls when [k] fails.
1185 Call the factory, install the result, hand it back; iteration, repr and everything else is inherited.
1186 Note the factory takes no arguments, which is why leaf recipes are zero-argument lambdas.
1187
1188 Counter -- count occurrences: a defaultdict(int) fold over the input. Missing keys read as 0 (mind the
1189 autovivification gotcha above); rank the .items() with sortOn.
1190 >>> c = Counter("abracadabra")
1191 >>> c['a'], c['z']
1192 (5, 0)
1193 >>> sortOn(lambda kv: -kv[1], list(Counter("aab").items()))
1194 [('a', 2), ('b', 1)]
1195
1196 Roll your own: a defaultdict(int) plus one fold -- d[x] += 1 per element. The += on a missing key is
1197 autovivification doing all the work.
1198
1199 insertWith -- Data.Map's insertWith f k v m: insert (k, v), combining with f(new, old) on collision --
1200 Haskell's argument order, new value first. PERSISTENT, also like Haskell: it returns a fresh dict and
1201 leaves the argument untouched.
1202 >>> insertWith(add, 'x', 4, {'x': 1})
1203 {'x': 5}
1204 >>> d = {'y': 2}
1205 >>> insertWith(add, 'x', 4, d)
1206 {'y': 2, 'x': 4}
1207 >>> d # the original survives
1208 {'y': 2}
1209
1210 Roll your own: one dict display -- {**d, k: ...}: the splat copies d (that IS the persistence), and the
1211 new key's value is f(v, d[k]) on collision (new value first, Haskell's order) or plain v. Compare
1212 fromListWith, which applies the same rule but mutates, because there the dict is its own private
1213 accumulator.
1214
1215 fromListWith -- THE dict-building fold: pour (key, value) pairs into a dict, combining collisions with
1216 f(new, old) -- Haskell's order, like insertWith. Keys keep first-seen order.
1217 >>> fromListWith(add, [(c, 1) for c in "aab"])
1218 {'a': 2, 'b': 1}
1219 >>> fromListWith(lambda new, old: old + new, [('a', [1]), ('b', [2]), ('a', [3])])
1220 {'a': [1, 3], 'b': [2]}
1221
1222 The order gotcha, faithfully Haskell's: because f receives (new, old), grouping with a bare concat
1223 REVERSES each group -- the classic Data.Map surprise. Combine as old + new to keep arrival order, or use
1224 plain addition where order cannot matter (counts).
1225 >>> fromListWith(add, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
1226 {2: ['ox', 'hi'], 3: ['sun']}
1227 >>> fromListWith(lambda new, old: old + new, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
1228 {2: ['hi', 'ox'], 3: ['sun']}
1229
1230 Why it earns its place -- one function, three monoids: choosing f is choosing what the dict MEANS.
1231 Addition over (k, 1) pairs counts; old + new over (k, [v]) pairs groups; max merges bags. That is the
1232 whole family: insertWith is one collision, fromListWith a list of them, unionWith two dicts' worth (where

```

```

1233 f receives (a's value, b's value)).
1234
1235 The recognition cue: whenever a loop builds a dict with an if-else on key existence, you are hand-rolling
1236 this fold. Name the pairs, name the monoid, and the loop disappears.
1237
1238 Remember it as: grouping, counting, and merging are one fold -- whose combiner hears the NEW value first.
1239
1240 Roll your own: an empty dict and one loop applying the collision rule per pair -- d[k] = f(v, d[k]) if
1241 the key exists, else v. It is insertWith inlined and IN-PLACE: mutation is fine here because the dict
1242 being built is the fold's own accumulator, invisible until returned.
1243
1244 unionWith -- merge two dicts, combining SHARED keys with f; a's keys keep their order, b's new keys
1245 follow. Choosing f is choosing a monoid: max gives bag union, + gives a summing merge.
1246 >>> unionWith(max, {'a': 1, 'b': 5}, {'b': 2, 'c': 3})
1247 {'a': 1, 'b': 5, 'c': 3}
1248 >>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4})
1249 {'a': 1, 'b': 5, 'c': 4}
1250
1251 Roll your own: two splats -- {**a, **{...}}: a copied whole, then overlaid with b's items after each has
1252 been through the collision rule, f(a[k], v) when k is shared, plain v when it is new. The asymmetry is
1253 deliberate: a's keys keep their positions, and only keys new to b append at the end.
1254
1255 group -- runs of CONSECUTIVE equal elements, as bare lists ([a], no keys) -- Data.List's group.
1256 >>> group("aaabbc")
1257 [['a', 'a', 'a'], ['b', 'b'], ['c']]
1258 >>> group("abab")
1259 [['a'], ['b'], ['a'], ['b']]
1260
1261 groupBy -- runs by an equivalence predicate; each newcomer is compared with eq against the run's FIRST
1262 element (span-based, exactly Haskell's groupBy), not against its neighbour -- a subtle and deliberate
1263 fidelity.
1264 >>> groupBy(lambda a, b: a == b, [1, 1, 2])
1265 [[1, 1], [2]]
1266 >>> groupBy(lambda a, b: b - a <= 1, [1, 2, 3, 10, 11])
1267 [[1, 2], [3], [10, 11]]
1268
1269 The word that matters is CONSECUTIVE: a new run starts whenever eq against the run's first element fails,
1270 so 'abab' yields four runs. Not a weakness -- a different question: runs, streaks and compression care
1271 about position, and these are the only tools asking about it.
1272 >>> [(run[0], len(run)) for run in group("aaabbbcc")] # run lengths
1273 [('a', 3), ('b', 4), ('c', 2)]
1274
1275 For CATEGORIES -- position irrelevant -- either sort first, so equals become adjacent, or use
1276 fromListWith, which never cared about position at all. Choosing between them is stating what your key
1277 means.
1278 >>> [(run[0], len(run)) for run in group(sorted("abab"))]
1279 [('a', 2), ('b', 2)]
1280
1281 Remember it as: group answers "how long are the runs?"; fromListWith answers "what are the piles?".
1282
1283 Roll your own: one pass in which each element either joins the current run or starts a fresh one. The
1284 test is eq(out[-1][0], x) -- against the run's FIRST element, not its last -- which is exactly how
1285 Haskell's span-based groupBy behaves, and the detail that makes non-transitive predicates (like "within 1
1286 of") agree with it.
1287
1288 bag_union -- multisets (bags) are Counters; union takes the LARGER count per key. Key order follows set
1289 union, so compare with == rather than printing.
1290 >>> bag_union(Counter("aab"), Counter("abb")) == {'a': 2, 'b': 2}
1291 True
1292
1293 bag_inter -- intersection: the SMALLER count per shared key, zero counts dropped. Note the implementation
1294 reads with .get, never [] -- indexing a Counter would autovivify zeros into it.
1295 >>> bag_inter(Counter("aab"), Counter("ab")) == {'a': 1, 'b': 1}
1296 True
1297 >>> bag_inter(Counter("aa"), Counter("bb")) == {}
1298 True
1299
1300 bag_diff -- difference, clipped at zero: what remains of a after removing b.
1301 >>> bag_diff(Counter("aab"), Counter("ab")) == {'a': 1}
1302 True
1303
1304 bag_sub -- inclusion: is bag a contained in bag b, multiplicities included? The ransom-note test.
1305 >>> bag_sub(Counter("ab"), Counter("aabb"))
1306 True
1307 >>> bag_sub(Counter("aab"), Counter("ab"))
1308 False
1309
1310 Tree -- nested defaultdicts to a FIXED depth, with a leaf factory at the bottom. Tree(1, list) is the
1311 grouping dict; Tree(2, int) is a sparse 2-D counting table. Indexing autovivifies the whole path.
1312 >>> t = Tree(1, list)
1313 >>> t['evens'].append(2)
1314 >>> t
1315 {'evens': [2]}
1316 >>> m = Tree(2, int)
1317 >>> m['a']['b'] += 1
1318 >>> m
1319 {'a': {'b': 1}}
1320

```

```

1321 ITree -- the INFINITE tree: every branch is another ITree, so any depth autovivifies on touch. This is
1322 the trie: index letter by letter and the branch builds itself.
1323 >>> t = ITree()
1324 >>> t['x']['y']['z'] = 9
1325 >>> t
1326 {'x': {'y': {'z': 9}}}
1327
1328 How it works: the definition is a knot -- ITree = lambda: defaultdict(ITree). Every missing key
1329 manufactures another ITree, which will do the same, without end. Indexing therefore GROWS the tree:
1330 touching t['c']['a']['t'] conjures the whole branch. A trie in one line, built from nothing but
1331 __missing__.
1332 >>> t = ITree()
1333 >>> for w in ["cat", "car"]: setpath(t, list(w) + ['$'], w)
1334 >>> sorted([leaf for p, leaf in paths(t) if p[-1] == '$'])
1335 ['car', 'cat']
1336
1337 Why it earns its place: hierarchies -- tries, directory trees, nested groupings -- stop needing setup
1338 code. Writing is indexing (or setpath, which also plants a leaf); harvesting is paths. The one edged
1339 rule: reads autovivify too, so QUESTIONS go through getpath, never bare indexing -- getpath's entry
1340 demonstrates the ghost branch this avoids.
1341
1342 Versus Tree(depth, leaf): Tree is the finite cousin -- fixed depth, with a real leaf factory (list, int)
1343 at the bottom instead of more tree. Tree(1, list) is the grouping dict; ITree is for depth you cannot
1344 know in advance, like words.
1345
1346 Remember it as: a dict that dreams up more of itself on demand.
1347
1348 Roll your own: ITree = lambda: defaultdict(ITree) mentions itself -- legal, because the lambda body only
1349 runs on a MISS, by which time the name is bound. Tree bottoms the same recursion out at a fixed depth by
1350 handing the last level a real leaf factory instead of more tree.
1351
1352 paths -- fold a whole Tree/ITree into a flat list of (keypath, leaf) pairs, in insertion order. The read-
1353 everything companion to setpath's write-one-path.
1354 >>> t = ITree()
1355 >>> setpath(t, ['a', 'b'], 1); setpath(t, ['a', 'c'], 2)
1356 >>> paths(t)
1357 [[['a', 'b'], 1], [['a', 'c'], 2]]
1358
1359 How it works: structural recursion with two arms, exactly the case-arms-as-fold story. A non-dict (or
1360 empty node) is a LEAF: report the single row ([], leaf). A dict is a BRANCH: recurse into every subtree
1361 and prepend the branching key to each keypath that comes back. The whole tree flattens into (keypath,
1362 leaf) rows, insertion-ordered, losslessly.
1363
1364 Why it earns its place: building a trie is the easy half; the value is HARVESTING it. autocomplete
1365 filters rows under a prefix, deepest_directory takes maxOn of keypath length, leaves reads the second
1366 column. One cata, many harvests -- you never write tree-walking code again.
1367 >>> t = ITree()
1368 >>> setpath(t, ['a', 'b'], 1); setpath(t, ['c'], 2)
1369 >>> paths(t)
1370 [[['a', 'b'], 1], [['c'], 2]]
1371 >>> maxOn(lambda row: len(row[0]), paths(t))
1372 [['a', 'b'], 1]
1373
1374 Remember it as: paths turns a tree back into rows; setpath turns rows into a tree.
1375
1376 Roll your own: recursion on one question -- is this still a dict? A leaf reports a single row with an
1377 empty path; a branch recurses into every child and conses its key onto every path that comes back. Depth
1378 equals tree height, comfortably inside Python's limit for sane tries.
1379
1380 leaves -- just the leaf values of paths.
1381 >>> t = ITree()
1382 >>> setpath(t, ['a'], 1); setpath(t, ['b'], 2)
1383 >>> leaves(t)
1384 [1, 2]
1385
1386 setpath -- write a leaf value at the end of a key path, autovivifying the branch on the way. Caution:
1387 writing at an interior key CLOBBERS the subtree below it.
1388 >>> t = ITree()
1389 >>> setpath(t, ['a', 'b'], 7)
1390 >>> t
1391 {'a': {'b': 7}}
1392 >>> setpath(t, ['a'], 0) # clobbers {'b': 7}
1393 >>> t
1394 {'a': 0}
1395
1396 Roll your own: walk all but the last key by PLAIN INDEXING -- on an ITree that autovivifies the branch as
1397 you go -- then assign at the final key, the single write. getpath is the mirror image with the safety
1398 catch on: isinstance and 'in' checks instead of indexing, so reading plants nothing.
1399
1400 getpath -- read along a key path WITHOUT autovivifying, returning a default when the path is absent. This
1401 is why it exists: reading a trie with [] would plant ghost branches.
1402 >>> t = ITree()
1403 >>> setpath(t, ['a', 'b'], 7)
1404 >>> getpath(t, ['a', 'b']), getpath(t, ['a', 'z'], default=0)
1405 (7, 0)
1406 >>> 'z' in t['a'] # no ghost branch was planted
1407 False
1408

```



```

1409 deque -- a double-ended queue built from two stacks, all four end operations amortised O(1) (elements
1410 migrate between stacks only when one runs dry). This is the BFS frontier; a plain list's pop(0) is O(n).
1411 >>> d = deque([1, 2, 3])
1412 >>> d.appendleft(0); d.append(4)
1413 >>> d.popleft(), d.pop()
1414 (0, 4)
1415 >>> len(d), list(d)
1416 (3, [1, 2, 3])
1417
1418 How it works: two plain lists, nose to nose -- _out holds the front (reversed), _in holds the back.
1419 append pushes onto _in, appendleft onto _out, both O(1). The trick is popping an EMPTY side: the other
1420 list is reversed across in one O(n) migration, and popping is O(1) again.
1421
1422 Why that still counts as O(1): amortization. Each element migrates at most once between being pushed and
1423 popped, so n operations cost O(n) in total -- constant per operation on average, which is what the
1424 workload's big-O cares about. A bare Python list cannot say this: pop(0) shifts every element, every
1425 single time.
1426 >>> q = deque()
1427 >>> for x in [1, 2, 3]: q.append(x)
1428 >>> q.popleft(), q.popleft() # first pop pays the migration; the next is free
1429 (1, 2)
1430
1431 Why it earns its place: BFS. The frontier must yield its OLDEST member (popleft) while newcomers join at
1432 the back (append) -- levelorder and grid_path are this class doing its one job.
1433
1434 Remember it as: two stacks facing each other make a queue; the reversal bill is paid once per element.
1435
1436 Roll your own: two plain lists nose to nose, each end appending and popping its own list at O(1). All the
1437 cleverness is in the pop guard: when your side is empty, reverse the OTHER list across in one go. Each
1438 element makes that crossing at most once between entering and leaving, so the occasional O(n) reversal
1439 amortises to O(1) per operation.
1440
1441 dsu -- disjoint-set union (union-find) over n elements, with path halving. find gives a set's
1442 representative; union merges and returns False when the two were ALREADY connected -- which is exactly a
1443 cycle detector (Kruskal's test).
1444 >>> find, union = dsu(4)
1445 >>> union(0, 1), union(1, 2)
1446 (True, True)
1447 >>> union(0, 2) # already connected: a cycle
1448 False
1449 >>> find(0) == find(2), find(0) == find(3)
1450 (True, False)
1451
1452 How it works: parent[] links every element toward its set's ROOT, and find follows the chain up. The
1453 subtle line is parent[x] = parent[parent[x]] -- path HALVING: every find re-points each visited node at
1454 its grandparent, so chains shrink as a side effect of being walked. That one line makes operations
1455 effectively constant time (inverse Ackermann, for the curious).
1456
1457 Why it earns its place: dynamic connectivity. "Are these two in the same group yet?" under a stream of
1458 merges is awkward for every other structure and trivial here. And union returning False -- already
1459 connected -- IS cycle detection: Kruskal's spanning tree is built by skipping every False.
1460 >>> find, union = dsu(5)
1461 >>> [union(a, b) for a, b in [(0, 1), (1, 2), (3, 4), (0, 2)]]
1462 [True, True, True, False]
1463 >>> len({find(i) for i in range(5)}) # two components remain
1464 2
1465
1466 Remember it as: find walks to the root, halving as it goes; a False union is a cycle caught.
1467
1468 Roll your own: parent[] as the forest and two closures instead of a class. find's loop carries the entire
1469 trick in one line -- parent[x] = parent[parent[x]] re-points every visited node at its grandparent (path
1470 halving), flattening chains as a side effect of walking them. union is find twice plus one root re-point,
1471 answering False when the roots already agree.
1472
1473 heappush -- push onto a min-heap kept in a plain list (sift-up). The smallest element is always h[0]. For
1474 a MAX-heap, push negated values.
1475 >>> h = []
1476 >>> for x in [5, 1, 8]: heappush(h, x)
1477 >>> h[0]
1478 1
1479
1480 How it works: the flat list IS a binary tree -- h[i]'s children live at 2i+1 and 2i+2 -- kept under one
1481 invariant: every parent <= its children. heappush appends at the first free leaf and sifts UP, swapping
1482 with its parent while smaller; heappop moves the last leaf to the root and sifts DOWN toward the smaller
1483 child. Each walks one root-to-leaf path: O(log n).
1484
1485 Why the invariant is enough: nothing is fully sorted -- siblings sit in any order -- yet h[0] is ALWAYS
1486 the minimum, because the root beats its children, who beat theirs. You pay log n per operation for a
1487 permanently readable minimum; draining the heap is heapsort; pushing negatives turns min into max.
1488 >>> h = []
1489 >>> for x in [7, 2, 9, 4]: heappush(h, x)
1490 >>> h[0]
1491 2
1492 >>> [heappop(h) for _ in range(4)]
1493 [2, 4, 7, 9]
1494
1495 Why it earns its place: "repeatedly take the best next thing" -- Dijkstra's frontier, running_median's
1496 two halves, any streaming top-k. When next-best is needed many times while the data keeps changing, the

```

```

1497 heap beats re-sorting every time.
1498
1499 Remember it as: a flat list, a parent-beats-children promise, and log-n repairs.
1500
1501 Roll your own: append at the first free leaf, then sift UP -- while the parent at (i - 1) // 2 is larger,
1502 swap and climb. The index arithmetic IS the tree; watch the array after four pushes:
1503 >>> h = []
1504 >>> for x in [7, 2, 9, 4]: heappush(h, x)
1505 >>> h
1506 [2, 4, 9, 7]
1507
1508 heappop -- remove and return the minimum (sift-down). Popping repeatedly drains in sorted order -- that
1509 is heapsort.
1510 >>> h = []
1511 >>> for x in [5, 1, 8]: heappush(h, x)
1512 >>> [heappop(h) for _ in range(3)]
1513 [1, 5, 8]
1514 >>> g = []
1515 >>> for x in [3, 9, 5]: heappush(g, -x)
1516 >>> -heappop(g)
1517 9
1518
1519 Roll your own: swap the root with the last leaf, pop it off the end, then sift DOWN -- repeatedly swap
1520 with the SMALLER of the two children (choosing the smaller is the classic bug site) until neither child
1521 is smaller. One root-to-leaf path each way: O(log n).
1522
1523 ===== 13. nodes =====
1524
1525 TreeNode -- the LeetCode binary tree: val, left, right, with None for absent children.
1526 >>> t = TreeNode(2, TreeNode(1), TreeNode(3))
1527 >>> t.left.val, t.val, t.right.val
1528 (1, 2, 3)
1529
1530 tfold -- the tree catamorphism: recursively combines each node's value with the already-folded left and
1531 right results, z standing in for empty subtrees. EVERY structural tree computation is one choice of f and
1532 z -- the five traversal/measure functions below are all tfold one-liners.
1533 >>> t = TreeNode(2, TreeNode(1), TreeNode(3))
1534 >>> tfold(lambda v, l, r: v + l + r, 0, t)
1535 6
1536 >>> tfold(lambda v, l, r: max(v, l, r), 0, t)
1537 3
1538
1539 How it works: the tree catamorphism -- structural recursion with a contract. Two arms, exactly like a
1540 case expression: the empty tree answers z; a node answers f(value, left_result, right_result), both
1541 results computed by the same recursion. Hand it (f, z) and any question about a tree becomes one line --
1542 the five traversals and both measures in section 13 are all tfold with different arms.
1543
1544 Why it earns its place: it separates WALKING from DECIDING. The recursion is written once, correct
1545 forever; you only ever author the arms. Even tree TRANSFORMATION fits -- mirror a tree by swapping the
1546 child results while rebuilding:
1547 >>> t = TreeNode(2, TreeNode(1), TreeNode(3))
1548 >>> mirror = tfold(lambda v, l, r: TreeNode(v, r, l), None, t)
1549 >>> inorder(mirror)
1550 [3, 2, 1]
1551
1552 The one caveat: recursion depth equals tree height -- fine for balanced and interview-sized trees; say a
1553 sentence out loud about adversarially skewed ones.
1554
1555 Remember it as: z is the empty-tree arm, f is the node arm -- the fold walks, you decide.
1556
1557 Roll your own: one line of structural recursion -- z when the tree is None, else f over the node's value
1558 and the two recursive results.
1559
1560 Roll your own: one line of structural recursion -- z when the tree is None, else f over the node's value
1561 and the two recursive results. Every traversal and measure in this section is that line with a different
1562 f plugged in.
1563
1564 inorder -- left, node, right; on a binary SEARCH tree this is sorted order.
1565 >>> inorder(TreeNode(2, TreeNode(1), TreeNode(3)))
1566 [1, 2, 3]
1567
1568 preorder -- node first, then children: the copy/serialise order.
1569 >>> preorder(TreeNode(2, TreeNode(1), TreeNode(3)))
1570 [2, 1, 3]
1571
1572 postorder -- children first, then node: the delete/evaluate order.
1573 >>> postorder(TreeNode(2, TreeNode(1), TreeNode(3)))
1574 [1, 3, 2]
1575
1576 tdepth -- the height of the tree.
1577 >>> tdepth(TreeNode(2, TreeNode(1), TreeNode(3)))
1578 3
1579
1580 tsize -- the number of nodes.
1581 >>> tsize(TreeNode(2, TreeNode(1), TreeNode(3)))
1582 3
1583
1584 levelorder -- breadth-first values, level by level: the deque earning its keep. Compare preorder on the

```

```

1585 same tree to see DFS vs BFS.
1586 >>> t = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
1587 >>> levelorder(t)
1588 [1, 2, 3, 4]
1589 >>> preorder(t)
1590 [1, 2, 4, 3]
1591
1592 Roll your own: seed a deque with the root, then loop -- popleft, record the value, append the children
1593 that exist. FIFO order IS breadth-first; swap the deque for a list used as a stack and the identical loop
1594 turns into depth-first.
1595
1596 ListNode -- the LeetCode singly-linked list: val and next.
1597 >>> n = ListNode(1, ListNode(2))
1598 >>> n.val, n.next.val
1599 (1, 2)
1600
1601 from_list -- build a linked list from a Python list: literally foldr(ListNode, xs, None). foldr's f takes
1602 (element, accumulator), and ListNode(val, nxt) has exactly that shape, so the last element is wrapped
1603 first and each earlier one points at the list built so far.
1604 >>> to_list(from_list([1, 2, 3]))
1605 [1, 2, 3]
1606
1607 to_list -- walk a linked list back into a Python list; the round-trip partner of from_list, and the way
1608 to make node results printable. An unfoldr: the seed is the node, each step yields (node.val, node.next),
1609 and None ends it.
1610 >>> to_list(from_list("ab"))
1611 ['a', 'b']
1612 >>> to_list(None)
1613 []
1614
1615 reverse_list -- reverse in place with the three-pointer walk; prev is a fold accumulator wearing
1616 pointers.
1617 >>> to_list(reverse_list(from_list([1, 2, 3])))
1618 [3, 2, 1]
1619
1620 Roll your own: the three-pointer walk compressed into one simultaneous assignment -- node.next, prev,
1621 node = prev, node, node.next. The right-hand side is evaluated in full before any assignment lands, which
1622 is what saves the pointer you are about to overwrite.
1623
1624 middle -- the middle node by fast/slow pointers (fast moves twice per step); on even lengths, the SECOND
1625 middle.
1626 >>> middle(from_list([1, 2, 3])).val
1627 2
1628 >>> middle(from_list([1, 2, 3, 4])).val
1629 3
1630
1631 Roll your own: fast advances two hops for slow's one, so when fast falls off the end, slow stands at the
1632 middle. The loop condition (fast and fast.next) is precisely what selects the SECOND middle on even
1633 lengths.
1634
1635 has_cycle -- Floyd's tortoise and hare: the fast pointer laps the slow one if and only if a cycle exists;
1636 O(1) space.
1637 >>> has_cycle(from_list([1, 2, 3]))
1638 False
1639 >>> n = ListNode(1); n.next = n # a self-loop
1640 >>> has_cycle(n)
1641 True
1642
1643 Roll your own: the same fast/slow walk, but watching for the pointers to MEET -- on a cycle the fast
1644 runner laps the slow one; on a straight list it simply falls off. O(1) space, no visited set.
1645
1646 merge_lists -- merge two SORTED linked lists, using the dummy-head idiom to avoid special-casing the
1647 first node.
1648 >>> to_list(merge_lists(from_list([1, 3]), from_list([2, 4])))
1649 [1, 2, 3, 4]
1650
1651 Roll your own: the dummy-head idiom -- a throwaway node to hang the result from, so appending the first
1652 real node needs no special case; walk both lists copying the smaller head, glue the survivor, return
1653 dummy.next.
1654
1655 ===== 14. grids & windows =====
1656
1657 neighbors4 -- the in-bounds orthogonal neighbours of cell (r, c) in an R-by-C grid. Bounds checking lives
1658 HERE, so BFS bodies stay clean.
1659 >>> sorted(neighbors4(0, 0, 2, 2)) # corner: two neighbours
1660 [(0, 1), (1, 0)]
1661 >>> sorted(neighbors4(1, 1, 3, 3)) # centre: four
1662 [(0, 1), (1, 0), (1, 2), (2, 1)]
1663
1664 Roll your own: yield each in-bounds delta of (r, c) -- the bounds check lives HERE so every caller's BFS
1665 body stays clean. neighbors8 is the same generator over a 3x3 delta grid, with (dr or dc) excluding the
1666 centre cell.
1667
1668 neighbors8 -- the same plus diagonals.
1669 >>> len(list(neighbors8(1, 1, 3, 3)))
1670 8
1671 >>> sorted(neighbors8(0, 0, 2, 2))
1672 [(0, 1), (1, 0), (1, 1)]

```

```

1673
1674 longest_window -- the sliding-window skeleton: it grows the right edge one element at a time (calling
1675 add), shrinks the left edge while your valid() says the window is illegal (calling shed), and tracks the
1676 best length. YOU supply the state as closures; the skeleton does the two-pointer bookkeeping. Longest-
1677 substring-without-repeats and longest-run-under-a- budget are both three closures away.
1678 >>> seen = []
1679 >>> longest_window("abcabb", lambda: len(seen) == len(set(seen)),
1680 ...               seen.append, lambda c: seen.pop(0))
1681 3
1682 >>> w = []
1683 >>> longest_window([2, 1, 3, 4], lambda: sum(w) <= 6,
1684 ...               w.append, lambda x: w.pop(0))
1685 3
1686
1687 How it works: inversion of control. The skeleton owns the two pointers -- it grows hi one element per
1688 step (calling add), then shrinks lo while your valid() reports the window broken (calling shed per
1689 evicted element) -- and it tracks the best length ever seen legal. You own only the STATE, held in
1690 closures over variables sitting next to the call.
1691
1692 Designing the closures is answering one question: "what makes a window ILLEGAL, and what must I track to
1693 know?" Distinct characters -> a duplicate count. A budget -> a running sum. At most k of something -> a
1694 counter. add and shed are that state's increment and decrement; valid is the legality test.
1695 >>> w = []
1696 >>> longest_window("aabcb", lambda: len(w) == len(set(w)), w.append, lambda c: w.pop(0))
1697 3
1698
1699 Why closures and not a class: the state lives exactly as long as the call, sits in scope where you can
1700 read it, and needs no ceremony. Functions carrying state -- the same muscle as curry's captured argument,
1701 here working a two-pointer job.
1702
1703 Remember it as: the skeleton slides the window; your three closures say what legal means.
1704
1705 Roll your own: hi sweeps forward exactly once, add() greeting each newcomer; an inner while shrinks lo,
1706 shed() by shed(), until valid() approves again. Every element is added once and removed at most once --
1707 the entire two-pointer O(n) argument, visible in two loops.
1708
1709 ===== 15. control =====
1710
1711 memo -- an unbounded memoizer as a plain decorator (no parentheses): results are cached by the argument
1712 tuple, so each distinct call computes once. Keyword arguments fold into the key; the cache is exposed as
1713 .cache.
1714 >>> @memo
1715 ... def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
1716 >>> fib(30)
1717 832040
1718 >>> fib.cache[(10,)]
1719 55
1720 >>> len(fib.cache)                                # one entry per distinct argument
1721 31
1722
1723 How it works: the decorator wraps f with a dict keyed on the arguments. First call computes and stores;
1724 every repeat is a lookup. That is the entire mechanism -- and why arguments must be HASHABLE: lists
1725 become tuples before they can be keys.
1726
1727 Why it earns its place: memoized recursion IS dynamic programming. Write the honest recurrence -- the
1728 answer for state (i, j) in terms of smaller states -- decorate it, and the cache is your DP table, filled
1729 in dependency order with no index bookkeeping. The exposed .cache lets you SEE the table: its size is the
1730 state count, exactly the number complexity analysis wants said out loud.
1731 >>> @memo
1732 ... def ways(r, c):                                # lattice paths to (r, c), moving right/down
1733 ...     return 1 if r == 0 or c == 0 else ways(r - 1, c) + ways(r, c - 1)
1734 >>> ways(3, 3)
1735 20
1736 >>> len(ways.cache)                                # the 4x4 grid minus (0,0), never needed
1737 15
1738
1739 Versus functools.lru_cache: the standard library's version adds eviction (maxsize); memo is deliberately
1740 unbounded, because DP wants every state kept -- and it is nine lines you can retype in any interview that
1741 bans imports.
1742
1743 Remember it as: recursion states the truth; the cache makes it affordable; .cache is the table.
1744
1745 Roll your own: a dict captured in a closure, keyed by the argument tuple (keyword arguments folded in as
1746 a frozenset when present -- tuples and frozensets because keys must be hashable). Compute on miss, look
1747 up on hit, and hang the dict on the wrapper as .cache so the DP table stays inspectable.
1748
1749 until -- iterate f from x until the predicate holds, returning the first satisfying value: a fixed-point
1750 loop with no recursion-depth limit. The state can be a tuple carrying whatever the loop must remember.
1751 >>> until(lambda x: x > 100, lambda x: x * 2, 1)
1752 128
1753 >>> collatz = lambda s: (s[0] // 2 if even(s[0]) else 3 * s[0] + 1, s[1] + 1)
1754 >>> until(lambda s: s[0] == 1, collatz, (6, 0))      # 8 steps to reach 1
1755 (1, 8)
1756
1757
1758 How it works: keep applying f until p approves -- iteration to a goal or a FIXED POINT, as a loop rather
1759 than recursion, so depth is unbounded. The state can be anything; a tuple lets one loop carry several
1760 facts at once, like the collatz example's (value, steps).

```

```
1761
1762 Why it earns its place: some processes have no list to fold over -- they run "until stable". Newton's
1763 isqrt converges; bellman_ford relaxes every edge until the distance map stops changing (the textbook
1764 fixed point; g7's hint calls it by name). until states such loops declaratively: here is the step, here
1765 is done.
1766 >>> until(lambda n: n * n >= 30, succ, 0)          # smallest n with n squared >= 30
1767 6
1768 >>> until(lambda xs: xs == sorted(xs), sorted, [3, 1, 2])
1769 [1, 2, 3]
1770
1771 Remember it as: while-not-done, promoted to an expression that returns its answer.
1772
1773 Roll your own: while not p(x): x = f(x); return x. Iteration promoted to a value-returning expression --
1774 and a loop rather than recursion on purpose, because a fixed point may take unboundedly many steps."""
1775
1776 from prelude import *                                # the one permitted import
1777
1778 add = lambda a, b: a + b                             # used throughout the examples
1779
1780
1781 if __name__ == u'__main__':
1782     import doctest
1783     r = doctest.testmod()
1784     end print(f"prelude-doctests: {r.attempted - r.failed}/{r.attempted} examples passing")
```